

DL-SIFA: Deep Learning Statistical Ineffective Fault Analysis on AES-128

CS6630 — Secure Processor Microarchitecture

Singh Shivam Ramdarash
CS25M048

Indian Institute of Technology Madras

May 2026

Outline

- 1 Introduction
- 2 Fault Model
- 3 Traditional SIFA
- 4 DL-SIFA
- 5 Comparison
- 6 Conclusion

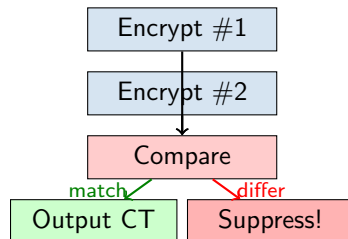
Background: Fault Attacks and Defenses

Fault Attacks:

- Inject physical faults during encryption
- Faulty output leaks the secret key
- DFA needs both correct and faulty ciphertexts

Defense — Temporal Redundancy:

- Encrypt the same plaintext **twice**, compare
- Outputs differ → fault detected → suppress
- Blocks DFA completely



SIFA: Breaking Temporal Redundancy

Key Insight (Dobraunig et al., TCHES 2018)

~50% of injected faults are *ineffective*—the target bit was already in the faulted state. These pass the redundancy check, but carry a **statistical bias** that leaks the key.

Project Goal:

- 1 Implement AES-128 with temporal redundancy + stuck-at-0 faults at Round 8
- 2 **Traditional SIFA**: recover key using bit-bias formula
- 3 **DL-SIFA**: recover key using a trained neural network
- 4 Compare trace efficiency of both methods

Based on: Ramezanzpour et al., “*Fault Intensity Map Analysis with NN Key Distinguisher*”, ASHES 2019.

Fault Model and Bias Mechanism

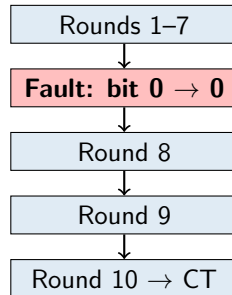
Fault	Stuck-at-0 (bit 0)
Location	Round 8 SubBytes input
Attempts	50,000 per byte
Ineffective rate	~50%
Traces kept	~25,000

Why bias?

Normal: $\text{byte} \in \{0 \dots 255\}$ (256 values)

After fault: only even values survive (128)

→ half the histogram bins are empty



Traditional SIFA: Bit-Bias Key Recovery

Algorithm: For each key guess $k \in \{0, \dots, 255\}$:

- 1 Invert each CT through rounds 10, 9, 8 using guess k
- 2 Compute bias = $\frac{\#(\text{bit } 0=0)}{N}$. Correct key $\rightarrow 1.0$, Wrong $\rightarrow 0.5$

Results:

- All 16 bytes: bias = 1.0000
- Recovered $K_{10}[0] = 0xD0$ ✓
- Correct bias: 1.0, avg wrong: 0.498
- **Min traces: 20**

Progressive recovery:

Traces	Result
10	FAIL
20	SUCCESS
50+	SUCCESS

DL-SIFA: Neural Network Approach

Motivation: Traditional SIFA needs exact fault model. In real hardware, faults are noisy and imprecise.

Profiling (500 random keys):

- 1 Collect faulted + clean CTs
- 2 Invert to R8, build **256-bin histogram**
- 3 Label: biased = 1, clean = 0
- 4 3,000 training samples

Attack (unknown key):

- 1 For each guess: invert, histogram, NN score
- 2 Highest score = recovered key

MLP Architecture:

Layer	Size	Details
Input	256	Histogram
FC+BN	128	ReLU, Drop 0.3
FC+BN	64	ReLU, Drop 0.2
FC	32	ReLU
Output	1	Sigmoid

BCE loss, Adam (lr=0.001)
200 epochs, PyTorch

Training: 100% accuracy

Key recovery:

- Recovered $K_{10}[0] = 0xD0$ ✓
- Correct score: **1.0000**
- Avg wrong score: 0.0001
- **Min traces: 50**

Stronger separation than traditional:
1.0 vs 0.0 (instead of 1.0 vs 0.5)

Progressive recovery:

Traces	Result
10	FAIL
20	FAIL
50	SUCCESS
100+	SUCCESS

With <50 traces, histogram is too noisy for the NN.

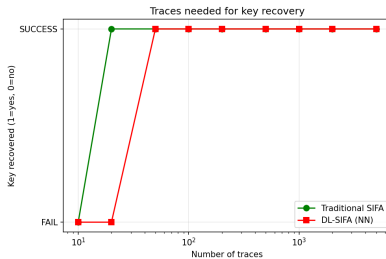
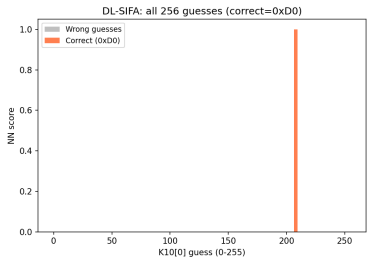
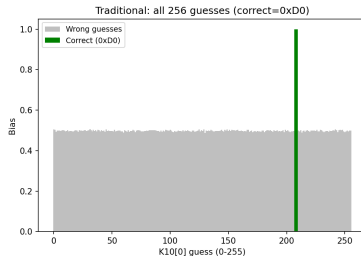
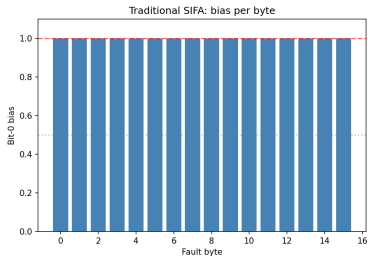
Comparison: Traditional vs DL-SIFA

Criterion	Traditional SIFA	DL-SIFA
Key recovered	✓ (0xD0)	✓ (0xD0)
Min traces	20	50
Separation	1.0 vs 0.5	1.0 vs 0.0
Needs fault model	Yes	No
Noise tolerance	No	Yes
Complexity	Low (formula)	High (training)

Traditional: fewer traces, simpler, but only works with known fault model

DL-SIFA: stronger signal, works without knowing fault details, more practical for real hardware

Results



Conclusion

- 1 SIFA breaks AES-128 with temporal redundancy using only **correct outputs**
- 2 Both methods recover $K_{10}[0] = 0xD0$
- 3 Traditional SIFA: **20 traces**, but needs exact fault model
- 4 DL-SIFA: **50 traces**, fault-model agnostic, better separation
- 5 Real-world faults are rarely known precisely $\rightarrow$ DL-SIFA is more practical

Future work: noisy faults, multi-bit faults, other ciphers, CNNs

References:

- 1 Dobraunig et al., “SIFA,” TCHES 2018
- 2 Ramezanpour et al., “Fault intensity map analysis with NN key distinguisher,” ASHES 2019

Thank You!

Questions?